

## TP 28 — Un classifieur complet

Semaine 28 ■ Fichiers dans NSI/TP28/ ■ 2 h

### Objectifs

- Implémenter KNN de bout en bout sur un jeu de données réel
- Évaluer le taux de réussite et choisir  $k$  expérimentalement
- Constater l'effet de la normalisation

Jeu de données : le classique « iris » (150 fleurs, 4 mesures, 3 espèces), fourni en CSV par le professeur (ou tout autre jeu étiqueté à attributs numériques). Tout ce TP réutilise l'année entière : CSV (S18), distances (S8), tri à clé (S20), occurrences (S10), tests (S13).

### Partie A — Préparer les données — `knn.py`

(30 min)

1. Charger `iris.csv`, convertir les 4 attributs en `float` (l'espèce reste une chaîne).
2. Mélanger la table (`random.shuffle`) puis la couper : 120 individus d'**entraînement**, 30 de **test** (tranches!).
3. `distance(e1, e2)` — distance euclidienne sur les 4 attributs (boucle sur une liste `ATTRIBUTS` des noms de champs). Tests : distance d'un individu à lui-même = 0 ; symétrie sur un exemple.

### Partie B — Le classifieur

(40 min)

1. `voisins(table, individu, k)` — les  $k$  plus proches (`sorted` + clé distance, tranche).
2. `classe_majoritaire(voisins)` — comptage + extremum (exercice 4).
3. `knn(table, individu, k)` — assembler.
4. Test artisanal : classer trois individus du jeu de test dont vous connaissez l'étiquette. Verdicts ?

### Partie C — Évaluation sérieuse

(35 min)

1. `taux_reussite(entrainement, test, k)` — classer chaque individu du jeu de test, comparer à l'étiquette réelle, renvoyer la proportion de succès.
2. Afficher le taux pour  $k \in \{1, 3, 5, 7, 11, 21, 51\}$ . Lequel gagne ? La dégradation aux grands  $k$  était-elle prévue par le cours ?
3. Re-mélanger les données et recommencer : les taux bougent-ils ? Moralité sur les petits jeux de test ?
4. ★ La matrice des confusions du pauvre : pour  $k$  choisi, afficher quel(le) espèce est confondue avec laquelle (dictionnaire des paires (réel, prédit)).

### Partie D — Défi ★★ — la normalisation

(15 min)

Multiplier artificiellement un attribut par 100 dans toute la table (déséquilibre d'échelle) : le taux s'effondre-t-il ? Écrire `normaliser(table)` (chaque attribut ramené dans  $[0, 1]$  — le `normalise` du TP 9 par colonne) et montrer que le taux remonte.